

รายงานสรุปการวิเคราะห์ ออกแบบ และพัฒนาระบบ

โครงการ: ระบบจัดการร้านกาแฟ (Cafe POS & Inventory System)

จัดทำโดย

1. นางสาวเมธิตา ทวีทรัพย์ รหัสนิสิต 67160195
2. นางสาวนนลนีย์ ผะลิวรรณ์ รหัสนิสิต 67160014
3. นายณฐกร โกศินานนท์ รหัสนิสิต 67160004
4. นายจิรภัทร บัวชุมสุข รหัสนิสิต 67160390
5. นายชนเมธ พิมพ์เบ้าทำ รหัสนิสิต 67160024

กลุ่ม: buumonkey

รายงานนี้เป็นส่วนหนึ่งของรายวิชา การวิเคราะห์และออกแบบระบบ (System Analysis and Design)

คณะวิทยาการสารสนเทศ มหาวิทยาลัยบูรพา

ภาคเรียนที่ 1 ปีการศึกษา 2569

คำนำ

รายงานฉบับนี้เป็นส่วนหนึ่งของรายวิชา การวิเคราะห์และออกแบบระบบ (System Analysis and Design) จัดทำขึ้นเพื่อรวบรวมและนำเสนอผลการศึกษา วิเคราะห์ และพัฒนาระบบจัดการร้านกาแฟ (Cafe POS & Inventory System) โดยครอบคลุมตั้งแต่การศึกษาความเป็นไปได้ การวิเคราะห์ความต้องการของผู้ใช้ การออกแบบสถาปัตยกรรมระบบ ฐานข้อมูล แผนภาพ UML ตลอดจนผลการทดสอบและการตรวจสอบความปลอดภัยของระบบ

การจัดทำรายงานเล่มนี้สำเร็จลุล่วงไปได้ด้วยดี ขอขอบพระคุณอาจารย์ผู้สอนรายวิชา ที่ได้ให้คำแนะนำแนวทางการดำเนินงาน และข้อคิดเห็นที่เป็นประโยชน์ตลอดภาคการศึกษา คณะผู้จัดทำหวังเป็นอย่างยิ่งว่ารายงานฉบับนี้จะเป็นประโยชน์และให้ความรู้แก่ผู้ที่สนใจศึกษาเกี่ยวกับการพัฒนาและออกแบบระบบสารสนเทศต่อไปหากรายงานฉบับนี้มีข้อผิดพลาดประการใด คณะผู้จัดทำขออภัยมา ณ ที่นี้ด้วย

คณะผู้จัดทำ (กลุ่ม buumonkey)

สารบัญ

คำนำ 2

บทที่ 1: บทนำ (Introduction) 5

1.1 บริบทและที่มาของโครงการ 5

1.2 สภาพปัญหาเดิม (Pain Points) 5

1.3 วัตถุประสงค์ของโครงการ 5

บทที่ 2: การวิเคราะห์ความเป็นไปได้ (Feasibility Analysis) 6

2.1 การประเมินความเป็นไปได้ 4 มิติ (TELOS Framework) 6

2.2 การเปรียบเทียบกระบวนการทำงาน (BPMN Analysis) 7

บทที่ 3: การวิเคราะห์ความต้องการ (Requirements Analysis) 8

3.1 สรุปผลการสัมภาษณ์ผู้มีส่วนได้ส่วนเสีย (Interview Summary) 8

3.2 ข้อกำหนดความต้องการทางฟังก์ชัน (Functional Requirements - FR) 8

3.3 ข้อกำหนดความต้องการที่ไม่ใช่ฟังก์ชัน (Non-functional Requirements - NFR) 8

3.4 ข้อจำกัดของระบบ (Constraints) 9

บทที่ 4: User Stories และ Use Case Diagram 9

4.1 ตาราง User Stories และ Acceptance Criteria (US-01 ถึง US-10) 9

4.2 สรุป Use Case Diagram และความสัมพันธ์ 10

บทที่ 5: การออกแบบเชิงวัตถุ (OOAD) 11

5.1 Class Diagram 11

5.2 Object Diagram (สถานการณ์ตัวอย่าง Order99) 11

บทที่ 6: การออกแบบฐานข้อมูล (Database Design) 12

6.1 ER Diagram และโครงสร้างตาราง (Relational Schema) 12

6.2 การรักษาความถูกต้องของข้อมูล (Data Integrity & Constraints) 13

บทที่ 7: การออกแบบกระบวนการและพฤติกรรมระบบ (Behavioral Design) 14

7.1 Sequence Diagram: กระบวนการสั่งซื้อและตัดสต็อก (Order Creation Flow) 14

บทที่ 8: การออกแบบ Input/Output และ UI 14

8.1 การออกแบบ Input Form (หน้า POS หน้าร้าน) 14

8.2 การออกแบบ Output (ใบเสร็จและรายงาน) 15

บทที่ 9: สถาปัตยกรรมระบบ (System Architecture) 15

9.1 รูปแบบสถาปัตยกรรมหลัก (3-Tier Layered Architecture) 15

9.2 การอธิบายตาม 4+1 View Model 16

บทที่ 10: ผลการทดสอบ Usability (Usability Testing) 17

10.1 วิธีการและสถานการณ์การทดสอบ 17

10.2 รายการ Usability Issues ที่พบและการแก้ไข 17

บทที่ 11: ผลการรวมระบบ (System Integration) 18
11.1 การทดสอบการทำงานเชื่อมโยงแบบ End-to-End (E2E Integration Flow) 18
บทที่ 12: ผลการตรวจสอบคุณภาพและความปลอดภัย (Quality & Security Review) 18
12.1 สรุปผลการตรวจ Code & Security Checklist (12 API Endpoints) 18
12.2 การแก้ไขความปลอดภัยที่ได้ดำเนินการแล้ว 19
บทที่ 13: สรุปผลและข้อเสนอแนะ (Conclusion & Recommendation) 19
13.1 สรุปผลการดำเนินโครงการ 19
13.2 บทเรียนที่ได้รับ (Lessons Learned) 19
13.3 ข้อจำกัดและแนวทางการพัฒนาต่อในอนาคต (Future Work) 20
บทที่ 14: ภาคผนวก (Appendix) 20
14.1 รายงานการใช้งานปัญญาประดิษฐ์ (AI Disclosure Summary Report) 20

บทที่ 1: บทนำ (Introduction)

1.1 บริบทและที่มาของโครงการ

ร้านกาแฟ **Buumonkey Cafe** เป็นธุรกิจร้านกาแฟที่เปิดให้บริการทั้งหมด 3 สาขา โดยตั้งอยู่ในพื้นที่โดยรอบมหาวิทยาลัย ซึ่งมีกลุ่มลูกค้าหลักเป็น นิสิต นักศึกษา และบุคลากรของมหาวิทยาลัย ด้วยลักษณะของลูกค้าที่เข้ามาใช้บริการพร้อมกันเป็นจำนวนมากในช่วงเวลาเร่งด่วน (Peak Time) เช่น ช่วงเช้าก่อนเข้าเรียน และช่วงพักกลางวัน ส่งผลให้ปริมาณออเดอร์หน้าร้านเพิ่มขึ้นสูงอย่างรวดเร็วในเวลาอันสั้น

1.2 สภาพปัญหาเดิม (Pain Points)

จากการศึกษากระบวนการทำงานเดิมของร้าน พบปัญหาสำคัญดังนี้:

1. **ความผิดพลาดในการรับออเดอร์และการสื่อสาร:** พนักงานแคชเชียร์จดออเดอร์ด้วยมือลงบนกระดาษ ทำให้เกิดปัญหาอ่านลายมือไม่ออก ออเดอร์ตกหล่น หรือส่งข้อมูลไปยังบาริสต้าผิดพลาด โดยเฉพาะออเดอร์ที่มีรายละเอียดซับซ้อน เช่น การปรับระดับความหวาน หรือการเพิ่มท็อปปิ้ง
2. **ความล่าช้าในการคิดเงินและคิวสะสม:** การคำนวณราคารวมและเงินทอนด้วยตนเองใช้เวลานาน ทำให้เกิดคิวสะสมหน้าร้าน และสูญเสียโอกาสในการขายช่วง Peak Time
3. **ความยุ่งยากในการทำบัญชีและสรุปยอดขาย:** เจ้าของร้านต้องใช้เวลา 30–45 นาทีต่อวันในการรวบรวมสลิปกระดาษเพื่อสรุปยอดขายประจำวันของแต่ละสาขา ทำให้ได้ข้อมูลล่าช้าและเสี่ยงต่อข้อผิดพลาดจากการคีย์ข้อมูล
4. **ขาดระบบการควบคุมสต็อกวัตถุดิบแบบเรียลไทม์:** พนักงานต้องทำการนับสต็อกวัตถุดิบด้วยตนเอง หลังปิดร้าน ทำให้ไม่ทราบปริมาณวัตถุดิบคงเหลือที่แท้จริงระหว่างวัน ส่งผลให้วัตถุดิบสำคัญ (เช่น เมล็ดกาแฟ, นมสด) หมดโดยไม่ทราบล่วงหน้า ทำให้เสียโอกาสในการขาย

1.3 วัตถุประสงค์ของโครงการ

1. เพื่อพัฒนาระบบ POS บริหารจัดการการขายหน้าร้านที่ช่วยให้แคชเชียร์บันทึกออเดอร์ เลือกรายละเอียดสินค้า คำนวณราคารวมสุทธิ และรับชำระเงินได้อย่างถูกต้องรวดเร็ว
2. เพื่อพัฒนาระบบตัดสต็อกวัตถุดิบแบบเรียลไทม์ตามสูตรเมนูทันทีที่มีการขาย พร้อมระบบแจ้งเตือนเมื่อวัตถุดิบใกล้หมด (Safety Stock)
3. เพื่อพัฒนาระบบคิวเครื่องดื่มสำหรับบาริสต้า และระบบออกใบเสร็จรับเงินดิจิทัลผ่าน QR Code เพื่อลดการใช้กระดาษ
4. เพื่อพัฒนาระบบรายงานสรุปยอดขาย กำไรสุทธิ และรายงานเปรียบเทียบยอดขายระหว่าง 3 สาขา สำหรับเจ้าของร้านใช้ประกอบการตัดสินใจทางธุรกิจ

บทที่ 2: การวิเคราะห์ความเป็นไปได้ (Feasibility Analysis)

2.1 การประเมินความเป็นไปได้ 4 มิติ (TELOS Framework)

1. ความเป็นไปได้ด้านเทคนิค (Technical Feasibility)

- **โครงสร้างเทคโนโลยี:** ระบบพัฒนาส่วน Backend ด้วย Node.js และ Express.js เชื่อมต่อกับระบบฐานข้อมูลเชิงสัมพันธ์ MySQL และพัฒนาส่วน Frontend ด้วย HTML5, CSS และ Vanilla JavaScript ที่รันบน Web Browser ผ่านอุปกรณ์แท็บเล็ต Android
- **ความพร้อมของทีมงาน:** แม้สมาชิกในทีมจะมีประสบการณ์ด้าน Node.js และ Express ในระดับปานกลางถึงน้อย แต่ตัวระบบถูกออกแบบภายใต้สถาปัตยกรรม MVC ที่ไม่ซับซ้อนเกินไป และมีกรอบเวลาพัฒนาแบบ Iterative/Agile ช่วยให้ทีมสามารถเรียนรู้และปรับปรุงโค้ดได้ระหว่าง Sprint
- **ฮาร์ดแวร์:** ใช้แท็บเล็ต Android ทั่วไปที่มีอยู่แล้วในร้าน ไม่จำเป็นต้องลงทุนซื้อเครื่อง POS ราคาแพงเฉพาะทาง

2. ความเป็นไปได้ด้านเศรษฐศาสตร์ (Economic Feasibility)

- **ประมาณการต้นทุน (Cost Estimation):**
  - ค่าพัฒนาและจัดทำระบบ: 30,000 บาท
  - ค่าอุปกรณ์แท็บเล็ตและอุปกรณ์เสริมเพิ่มเติม (3 สาขา): 30,000 บาท
  - **งบประมาณรวมทั้งสิ้น:** 60,000 บาท
- **ผลตอบแทนและการคุ้มทุน (Financial Return):**
  - ประหยัดเวลาในการปิดยอดบัญชีประจำวัน 30–45 นาที/วัน/สาขา
  - ลดต้นทุนแฝงจากการสูญหายและหมดอายุของวัตถุดิบได้ประมาณ 10%
  - เพิ่มความรวดเร็วในการบริการ รองรับลูกค้านักศึกษาช่วง Peak Time ได้เพิ่มขึ้น 15%

3. **ระยะเวลาคืนทุน (Payback Period):** คาดว่าจะสามารถคืนทุนได้ภายใน **3 – 4 เดือน**ความเป็นไปได้ด้านการปฏิบัติงาน (Operational Feasibility)

- **การปรับตัวของผู้ใช้งาน:** เปลี่ยนจากการจดออเดอร์กระดาษมาใช้ระบบ POS บนหน้าจอแท็บเล็ต ออกแบบหน้าจอตามหลัก Usability ปุ่มกดมีขนาดใหญ่ จัดหมวดหมู่ชัดเจน
- **การฝึกอบรม:** จัดแผนฝึกอบรมการใช้งานระบบให้กับพนักงานแคชเชียร์และบาริสต้าเป็นเวลา 1 สัปดาห์ก่อนเปิดใช้งานจริง

4. **การสนับสนุนจากองค์กร:** เจ้าของร้านให้การสนับสนุนเต็มที่ เนื่องจากระบบตอบโจทย์การดูรายงานและควบคุมสต็อกแบบเรียลไทม์ความเป็นไปได้ด้านระยะเวลา (Schedule Feasibility)

โครงการกำหนดระยะเวลาดำเนินการรวม 16 สัปดาห์ แบ่งเป็น 4 ระยะ:

- **สัปดาห์ที่ 1 – 4:** การวิเคราะห์และออกแบบระบบ (Requirements, BPMN, UML, DB Design)
- **สัปดาห์ที่ 5 – 12:** การพัฒนาระบบ (แบ่งเป็น 3 Coding Sprints ตามหลัก Agile)
- **สัปดาห์ที่ 13 – 14:** การรวมระบบ ทดสอบ Usability และตรวจทานความปลอดภัย
- **สัปดาห์ที่ 15 – 16:** การสรุปผล จัดทำเอกสาร และนำเสนอโครงการ

2.2 การเปรียบเทียบกระบวนการทำงาน (BPMN Analysis)

กระบวนการเดิม (As-Is Process)

1. ลูกค้าเดินมาสั่งออเดอร์ที่เคาน์เตอร์
2. พนักงานแคชเชียร์เขียนรายการลงกระดาษ และคำนวณราคารวมด้วยเครื่องคิดเลข
3. แคชเชียร์รับเงินสด และเขียนเงินทอนลงในกระดาษ
4. แคชเชียร์เดินนำกระดาษออเดอร์ไปเสียบไว้ที่เคาน์เตอร์บาริสต้า
5. บาริสต้าอ่านลายมือบนกระดาษ และลงมือชงเครื่องดื่ม
6. เมื่อปิดร้าน พนักงานต้องมานั่งนับวัตถุดิบในคลังและลงบันทึกในสมุด *(ข้อเสีย: ทำงานซ้ำ ซ้อน ช้า เกิด Human Error สูง ไม่รู้สต็อกทันที)*

กระบวนการใหม่ (To-Be Process)

1. ลูกค้าสั่งออเดอร์ พนักงานแคชเชียร์เลือกเมนูและท็อปปิ้งบนหน้าจอ POS แท็บเล็ต
2. ระบบคำนวณราคารวมสุทธิให้อัตโนมัติทันที
3. แคชเชียร์ระบุการรับชำระเงิน (เงินสด/สแกน QR Code)
4. เมื่อชำระเงินสำเร็จ ระบบจะทำงานให้อัตโนมัติ (Service Tasks):
    - ทำการตัดสต็อกวัตถุดิบตามสูตรเมนูในฐานข้อมูลทันที
    - ส่งข้อมูลออเดอร์ไปยังหน้าจอแสดงคิวของบาริสต้า
    - แสดง QR Code ใบเสร็จดิจิทัลบนหน้าจอให้ลูกค้าสแกน
5. เจ้าของร้านสามารถเปิดดูรายงานสรุปยอดขายและสต็อกคงเหลือได้ตลอดเวลาแบบ Real-time

---

บทที่ 3: การวิเคราะห์ความต้องการ (Requirements Analysis)

3.1 สรุปผลการสัมภาษณ์ผู้มีส่วนได้ส่วนเสีย (Interview Summary)

จากการสัมภาษณ์เจ้าของร้าน พนักงานแคชเชียร์ และบาริสต้า สรุปความต้องการสำคัญได้ดังนี้:

- **เจ้าของร้าน:** ต้องการระบบที่สามารถดูยอดขายรวม สรุปรายรับ-รายจ่าย รายงานกำไรสุทธิ และเปรียบเทียบยอดขายระหว่าง 3 สาขาได้ ตรวจสอบสต็อกวัตถุดิบได้เรียลไทม์ โดยไม่ต้องลงทุนซื้ออุปกรณ์ฮาร์ดแวร์ราคาสูง
- **แคชเชียร์:** ต้องการหน้าจอ POS ที่กดง่าย มีรายการท็อปปิ้ง/ระดับความหวานให้เลือกชัดเจน คำนวณราคารวมและเงินทอนอัตโนมัติ ออกใบเสร็จได้รวดเร็ว
- **บาริสต้า:** ต้องการหน้าจอแสดงรายการออเดอร์ที่เรียงตามลำดับเวลา อ่านง่าย ไม่ต้องเดินไปหยิบกระดาษออเดอร์

3.2 ข้อกำหนดความต้องการทางฟังก์ชัน (Functional Requirements - FR)

- **FR-01 (Take & Record Order):** ระบบต้องสามารถบันทึกออเดอร์และเลือกเมนูเครื่องดื่มลงตะกร้าสินค้าได้
- **FR-02 (Add Toppings/Options):** ระบบต้องรองรับการเลือกปรับระดับความหวาน และเพิ่มท็อปปิ้ง/ออปชันเสริม พร้อมคำนวณราคาเพิ่มทันที
- **FR-03 (Calculate Total Price):** ระบบต้องคำนวณราคารวมและยอดสุทธิให้อัตโนมัติเมื่อมีการเปลี่ยนแปลงรายการในตะกร้า
- **FR-04 (Process Payment):** ระบบต้องรองรับการบันทึกการชำระเงินด้วยเงินสด (คำนวณ เงินทอน) และการสร้าง QR Code สำหรับสแกนชำระ
- **FR-05 (Issue Digital Receipt):** ระบบต้องแสดง QR Code บนหน้าจอ เพื่อให้ลูกค้าสแกนรับใบเสร็จดิจิทัลได้
- **FR-06 (Manage Inventory & Stock):** ระบบต้องตัดจำนวนสต็อกวัตถุดิบตามสูตรอัตโนมัติเมื่อชำระเงินสำเร็จ และแสดงข้อความเตือนเมื่อสต็อกต่ำ กว่าเกณฑ์ (Safety Stock)
- **FR-07 (Order Queue & Display):** ระบบต้องแสดงรายการออเดอร์ในคิวเรียงตามเวลา พร้อมรายละเอียดท็อปปิ้ง บนหน้าจอของบาริสต้า
- **FR-08 (Financial Report):** ระบบต้องสรุปรายงานรายรับ-รายจ่าย ยอดขาย และกำไรสุทธิตามช่วงเวลา (รายวัน/รายเดือน/รายปี) สำหรับเจ้าของร้านได้
- **FR-09 (Multi-branch Comparison Report):** ระบบต้องรองรับการแสดงรายงานเปรียบเทียบยอดขายระหว่างสาขาได้

3.3 ข้อกำหนดความต้องการที่ไม่ใช่ฟังก์ชัน (Non-functional Requirements - NFR)

- **NFR-01 (Performance):** ระบบต้องคำนวณราคา ประมวลผลการชำระเงิน และส่งข้อมูลเข้าคิวบาริสต้าภายในเวลา 5 – 10 วินาที
- **NFR-02 (Availability):** ระบบต้องมีความพร้อมใช้งานไม่น้อยกว่า 99% ของเวลาทำการของร้าน
- **NFR-03 (Usability / Portability):** ตัวระบบต้องรองรับการแสดงผลและใช้งานผ่านแท็บเล็ต Android หน้าร้านได้อย่างสะดวก

- **NFR-04 (Security):** ระบบต้องทำการเข้ารหัสรหัสผ่านผู้ใช้งานทุกคนด้วยเทคนิค bcrypt ก่อนจัดเก็บลงฐานข้อมูล

3.4 ข้อจำกัดของระบบ (Constraints)

1. พัฒนาส่วน Backend ด้วย Node.js / Express และใช้ MySQL เป็นระบบจัดการฐานข้อมูล
2. แสดงผลระบบหน้าร้านผ่านเว็บเบราว์เซอร์บนแท็บเล็ต Android
3. กรอบระยะเวลาในการพัฒนารวมทั้งสิ้น 3 - 4 เดือน (16 สัปดาห์)

บทที่ 4: User Stories และ Use Case Diagram

4.1 ตาราง User Stories และ Acceptance Criteria (US-01 ถึง US-10)

ID	Role (Actor)	User Story (As a... I want... So that...)	Acceptance Criteria (Given-When-Then)
US-01	แคชเชียร์	As a แคชเชียร์I want บันทึกออเดอร์และเลือกเมนูจากหน้าจอได้ So that รับรายการได้อย่างถูกต้องและรวดเร็ว	Given อยู่ที่หน้าสั่งซื้อ When กดเลือกเมนูเครื่องดื่ม Then ระบบเพิ่มรายการลงตะกร้าทันที
US-02	แคชเชียร์	As a แคชเชียร์I want เลือกเพิ่มท็อปปิง / ออปชันเสริมได้So that ตอบสนองความต้องการเฉพาะของลูกค้าได้	Given เลือกเมนูแล้ว When กดเลือกท็อปปิง Then ระบบบันทึกรายละเอียดและปรับเพิ่มราคาตามท็อปปิงทันที
US-03	แคชเชียร์	As a แคชเชียร์I want ให้ระบบคำนวณราคารวมและยอดสุทธิให้อัตโนมัติSo that	Given มีสินค้าในตะกร้า When ระบบรวมรายการ Then แสดงราคารวมสุทธิอัตโนมัติภายใน 5 วินาที
US-04	แคชเชียร์	As a แคชเชียร์I want ดำเนินการรับชำระเงินทั้งเงินสดและ QR CodeSo that ปิดออเดอร์และส่งรายการเข้าสู่ระบบได้	Given เลือกชำระเงินสด ยอด 80 รับ 100 When ระบุเงินรับ Then ระบบแสดงเงินทอน 20 บาททันที

ID	Role (Actor)	User Story (As a... I want... So that...)	Acceptance Criteria (Given-When-Then)
US-05	ลูกค้า	As a ลูกค้าI want ชำระเงินผ่านช่องทางที่สะดวกSo that ซื้อสินค้าได้สะดวกรวดเร็ว	Given เลือกชำระ QR When สแกน ชำระเงินสำเร็จ Then ระบบยืนยันการชำระเงินบนหน้าจอ
US-06	เจ้าของร้าน	As a เจ้าของร้านI want ให้ระบบตัดสต็อกให้อัตโนมัติและแจ้งเตือนเมื่อสต็อกต่ำ So that ทราบสต็อกจริงและป้องกันสินค้าขาด	Given ชำระเงินสำเร็จ When ออเดอร์บันทึก Then ระบบตัดสต็อกอัตโนมัติและเตือนหากสต็อกต่ำ กว่าเกณฑ์
US-07	ลูกค้า	As a ลูกค้าI want รับใบเสร็จรับเงินดิจิทัลผ่าน QR CodeSo that ตรวจสอบรายการได้โดยไม่ต้องใช้กระดาษ	Given ชำระเงินสำเร็จ When ระบบประมวลผลเสร็จ Then แสดง QR Code บนหน้าจอให้ลูกค้าสแกน
US-08	เจ้าของร้าน	As a เจ้าของร้านI want ดูรายการยอดขายเปรียบเทียบแบบหลายสาขา So that วิเคราะห์และเปรียบเทียบประสิทธิภาพแต่ละสาขาได้	Given เลือกช่วงเวลาและสาขา When กดประมวลผล Then ระบบแสดงกราฟ/ตารางเปรียบเทียบยอดขายระหว่างสาขา
US-09	บาริสต้า	As a บาริสต้าI want ดูรายการออเดอร์ที่ต้องชงในคิวตามที่ระบบแจ้งSo that ทำเครื่องดื่มได้ถูกต้องตามลำดับ	Given มีออเดอร์ชำระเงินแล้ว When ดูหน้าจอคิว Then ระบบแสดงรายการเครื่องดื่มพร้อมท็อปปิ้งเรียงตามเวลา
US-10	เจ้าของร้าน	As a เจ้าของร้านI want เข้าดูรายงานสรุปรายรับ-รายจ่ายและกำไรสุทธิSo that ประเมินผลประกอบการและวางแผนธุรกิจได้	Given ล็อกอินสิทธิ์เจ้าของร้าน When เข้าเมนูรายงาน Then แสดงสรุปรายรับ รายจ่ายและกำไรสุทธิตามช่วงเวลาที่เลือก

4.2 สรุป Use Case Diagram และความสัมพันธ์

- **Actors ในระบบ:**
  1. **แคชเชียร์ (Cashier):** มีปฏิสัมพันธ์กับ Use Case รับและบันทึกออเดอร์, คำนวณราคารวม สุทธิ, รับชำระเงิน, ออกใบเสร็จดิจิทัล
  2. **บาริสต้า (Barista):** มีปฏิสัมพันธ์กับ Use Case ดูออเดอร์ที่ตองชง, เช็กสต็อกวัตถุดิบ
  3. **เจ้าของร้าน (Owner):** มีปฏิสัมพันธ์กับ Use Case ดูรายงานยอดขาย, จัดการเมนู/ราคา, ดูสต็อก
- **ความสัมพันธ์ทางเทคนิค:**
  - รับและบันทึกออเดอร์ <<extend>> เพิ่มท็อปปิง (เกิดขึ้นเฉพาะเมื่อลูกค้าขอเพิ่มออปชัน)
  - รับและบันทึกออเดอร์ <<include>> คำนวณราคา (ต้องเกิดขึ้นเสมอทุกครั้งที่สั่งซื้อ)
  - คำนวณราคา <<include>> รับชำระเงิน

- รับชำระเงิน **<<include>>** ออกใบเสร็จ

บทที่ 5: การออกแบบเชิงวัตถุ (OOAD)

5.1 Class Diagram

Class Diagram ของระบบประกอบด้วยคลาสหลัก 4 คลาส ซึ่งสะท้อนโครงสร้าง Entity ทางธุรกิจ:

2. **คลาส Cashier:** เก็บข้อมูลพนักงานแคชเชียร์ (cashierId, name) มีความสัมพันธ์แบบ **Directed Association (**1 ⟶ 0.. ***)** ไปยังคลาส Order ในฐานะผู้ประมวลผลออเดอร์
3. **คลาส Order:** เก็บข้อมูลหัวบิลออเดอร์ (orderId, orderDate, paymentMethod, totalAmount) มีเมธอด getTotal() และ addItem()
4. **คลาส OrderItem:** เก็บรายละเอียดรายการสินค้าในบิล (quantity, unitPrice) มีเมธอด getSubtotal()
    - **ความสัมพันธ์ระหว่าง Order และ OrderItem เป็นแบบ Composition** OrderItem ดำรงอยู่ได้เฉพาะภายใน Order เท่านั้น หากลบ Order รายการ OrderItem ทั้งหมดจะถูกทำลายไปพร้อมกัน
5. **คลาส MenuItem:** เก็บข้อมูลเมนูสินค้า (menuId, name, price)
    - **ความสัมพันธ์ระหว่าง OrderItem และ MenuItem เป็นแบบ Aggregation** OrderItem อ้างอิงไปยัง MenuItem โดยที่ MenuItem เป็นอิสระ หากลบ Order/OrderItem ตัวเมนูหลักในระบบยังคงอยู่

หลักการ Price Snapshotting
ในคลาส OrderItem มีการกำหนด attribute unitPrice เพื่อคัดลอกราคา ณ เวลาที่เกิดการสั่งซื้อจริงมา จาก MenuItem.price การออกแบบนี้เรียกว่า **Price Snapshotting** มีวัตถุประสงค์เพื่อรักษาความถูกต้องของประวัติยอดขายในอดีต หากในอนาคตเจ้าของร้านทำการปรับเปลี่ยนราคาขายใน MenuItem ยอดขายรวมของออเดอร์ที่เคยเกิดขึ้นไปแล้วจะไม่ถูกกระทบ

5.2 Object Diagram (สถานการณ์ตัวอย่าง Order99)

- **สถานการณ์:** ลูกค้าสั่ง Iced Milk Tea 2 แก้ว, Lemonade 1 แก้ว และสั่ง Iced Milk Tea เพิ่มอีก 3 แก้ว รวมเป็นเงิน 355.00 บาท ชำระด้วยเงินสด โดยแคชเชียร์ "Nattida" (cashier2) เป็นผู้รับออ เดอร์
- **การจำลอง Instance ใน Object Diagram:**
    - Instance cashier2: Cashier (cashierId = "CS-002", name = "Nattida")
    - Instance order99: Order (orderId = "ORD-099", orderDate = "2025-05-18 10:30:00", paymentMethod = "Cash", totalAmount = 355.00)

- Instance itemA: OrderItem (quantity = 2, unitPrice = 55.00, subTotal = 110.00) ⟶ Aggregation ไปยัง menu10: MenuItem ("Iced Milk Tea")
- Instance itemB: OrderItem (quantity = 1, unitPrice = 80.00, subTotal = 80.00) ⟶ Aggregation ไปยัง menu15: MenuItem ("Lemonade")
- Instance itemC: OrderItem (quantity = 3, unitPrice = 55.00, subTotal = 165.00) ⟶ Aggregation ไปยัง menu10: MenuItem ("Iced Milk Tea")

- **ผลสรุป:** ยอดรวม 110. 00 + 80. 00 + 165. 00 = 355. 00 บาท ตรงตามเงื่อนไขทางธุรกิจ

บทที่ 6: การออกแบบฐานข้อมูล (Database Design)

6.1 ER Diagram และโครงสร้างตาราง (Relational Schema)

ฐานข้อมูลได้รับการออกแบบบน MySQL ชื่อฐานข้อมูล buumonkey_cafe ประกอบด้วย 5 ตารางหลัก:

```
SQL
CREATE DATABASE IF NOT EXISTS buumonkey_cafe;
USE buumonkey_cafe;

-- 1. ตารางสาขา (branch)
CREATE TABLE branch (
    branch_id VARCHAR(50) PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    address VARCHAR(255)
);

-- 2. ตารางพนักงานแคชเชียร (cashier)
CREATE TABLE cashier (
    cashier_id VARCHAR(50) PRIMARY KEY,
    branch_id VARCHAR(50) NOT NULL,
    name VARCHAR(100) NOT NULL,
    CONSTRAINT fk_cashier_branch FOREIGN KEY (branch_id) REFERENCES
branch(branch_id)
);

-- 3. ตารางรายการเมนูและสต็อก (menu_item)
CREATE TABLE menu_item (
    menu_id VARCHAR(50) PRIMARY KEY,
    branch_id VARCHAR(50) NOT NULL,
    name VARCHAR(100) NOT NULL,
    price DECIMAL(10,2) NOT NULL,
    stock_quantity INT NOT NULL DEFAULT 0,
```

```
    CONSTRAINT fk_menu_item_branch FOREIGN KEY (branch_id) REFERENCES
branch(branch_id),
    CONSTRAINT chk_menu_item_price CHECK (price >= 0),
    CONSTRAINT chk_menu_item_stock CHECK (stock_quantity >= 0)
);

-- 4. ตารางหัวบิลออเดอร์ (orders)
CREATE TABLE orders (
    order_id INT AUTO_INCREMENT PRIMARY KEY,
    branch_id VARCHAR(50) NOT NULL,
    cashier_id VARCHAR(50) NOT NULL,
    order_date DATETIME NOT NULL,
    payment_method VARCHAR(30) NOT NULL,
    CONSTRAINT fk_orders_branch FOREIGN KEY (branch_id) REFERENCES
branch(branch_id),
    CONSTRAINT fk_orders_cashier FOREIGN KEY (cashier_id) REFERENCES
cashier(cashier_id)
);

-- 5. ตารางรายการสินค้าในออเดอร์ (order_item)
CREATE TABLE order_item (
    order_item_id INT AUTO_INCREMENT PRIMARY KEY,
    order_id INT NOT NULL,
    menu_id VARCHAR(50) NOT NULL,
    quantity INT NOT NULL,
    unit_price DECIMAL(10,2) NOT NULL,
    customizations JSON NULL,
    CONSTRAINT fk_order_item_order FOREIGN KEY (order_id) REFERENCES
orders(order_id),
    CONSTRAINT fk_order_item_menu FOREIGN KEY (menu_id) REFERENCES
menu_item(menu_id),
    CONSTRAINT chk_order_item_quantity CHECK (quantity > 0),
    CONSTRAINT chk_order_item_unit_price CHECK (unit_price >= 0)
);
```

6.2 การรักษาความถูกต้องของข้อมูล (Data Integrity & Constraints)

- **Foreign Key Constraints:** เชื่อมโยงความสัมพันธ์ระหว่างตารางอย่างสมบูรณ์ ป้องกันข้อมูลขยะ (Orphan Data)
- **Check Constraints:** บังคับให้ price >= 0, stock_quantity >= 0 และ quantity > 0 เพื่อป้องกันข้อมูลการเงินหรือจำนวนติดลบ
- **JSON Field:** คอลัมน์ customizations ในตาราง order_item ถูกออกแบบให้จัดเก็บข้อมูลตัวเลือกเสริม (เช่น ความหวาน, ท็อปปิ้ง) ในรูปแบบ JSON Structure ช่วยให้ระบบมีความยืดหยุ่นโดยไม่ต้องสร้างตารางย่อยเพิ่มขึ้นซับซ้อน

บทที่ 7: การออกแบบกระบวนการและพฤติกรรมระบบ (Behavioral Design)

7.1 Sequence Diagram: กระบวนการสั่งซื้อและตัดสต็อก (Order Creation Flow)

ลำดับการทำงานระหว่างวัตถุเมื่อพนักงานทำรายการสั่งซื้อหน้าร้าน:

1. **User (แคชเชียร์) ⟶ POS UI:** เลือกรายการเมนู ระบุจำนวน ปรับแต่งท็อปปิ้ง แล้วกด [ยืนยันออ เดอร์]
2. **POS UI ⟶ OrderController:** ส่งคำขอแบบ POST /api/orders พร้อม Payload JSON (รายการสินค้า, paymentMethod, cashierId, branchId)
3. **OrderController (Validation):**
    - *Alternative 1 (Data Invalid):* หากข้อมูลไม่ถูกต้อง ตอบกลับ 400 Bad Request แสดงข้อความเตือนบนหน้าจอ
    - *Alternative 2 (Data Valid):* ดำเนินการตรวจสอบสต็อก
4. **OrderController ⟶ MenuItemModel:** เรียกฟังก์ชัน findManyForStockCheck(menuIds, branchId)
5. **MenuItemModel ⟶ MySQL DB:** ส่งคำสั่ง SELECT เช็กปริมาณ stock_quantity ตาม branch_id
    - *Alternative (Stock ไม่พอ):* หากสินค้าหมด ตอบกลับ 400 Bad Request แจ้งว่าวัตถุดิบไม่เพียงพอ
6. **OrderController ⟶ OrderModel (Database Transaction):**
    - สั่งสร้างหัวบิลออเดอร์ create(branchId, cashierId, paymentMethod, orderDate) ⟶ INSERT INTO orders
    - สั่งวนลูปบันทึกรายการสินค้า addItem(orderId, menuId, quantity, price) ⟶ INSERT INTO order_item
    - สั่งตัดสต็อกสินค้า deductStock(menuId, quantity, branchId) ⟶ UPDATE menu_item
7. **OrderController ⟶ POS UI:** ตอบกลับสถานะ 201 Created พร้อมส่งรายการ lowStockMenuIds (ถ้ามี)
8. **POS UI ⟶ User:** แสดงใบเสร็จรับเงินดิจิทัลบนหน้าจอ

บทที่ 8: การออกแบบ Input/Output และ UI

8.1 การออกแบบ Input Form (หน้า POS หน้าร้าน)

- **โครงสร้างหน้าจอ:** แบ่งออกเป็น 2 ฝั่ งหลัก
    - **ฝั่ งซ้าย (Menu Catalog):** ตัวเลือกหมวดหมู่แท็บ (ทั้งหมด, กาแฟ, ชา & อื่นๆ) และการ์ดเมนูแบบ Grid Layout แสดงชื่อเมนู ราคา และจำนวนสต็อกคงเหลือ (หากสต็อกต่ำ กว่า 10 จะแสดงป้ายสีส้ม "เหลือน้อย")
    - **ฝั่ งขวา (Order Cart):** ตะกร้าสินค้า แสดงรายการที่เลือก ปุ่มปรับเพิ่ม/ลดจำนวน (+/-) ปุ่ม ลบ ยอดรวมสุทธิ (TOTAL) แบบ Real-time และ Radio Button เลือกช่องทางชำระเงิน (เงินสด/QR Code)
- **Validation (Defense in Depth):**
    - *Client-side:* ป้องกันไม่ให้กดกดยืนยันออเดอร์หากไม่มีสินค้าในตะกร้า

- *Server-side:* ตรวจสอบพารามิเตอร์ผ่าน API และใช้ Strict Idempotency Check ป้องกันการส่งออเดอร์ซ้ำ

8.2 การออกแบบ Output (ใบเสร็จและรายงาน)

1. **ใบเสร็จรับเงินดิจิทัล (Digital Receipt Template):**
   - แสดงชื่อร้าน "BUUMONKEY CAFE", TAX ID, เลขที่สาขา, เลขที่ใบเสร็จ, ชื่อแคชเชียร์, วันที่/เวลา
   - ตารางรายการสินค้า จำนวน ราคาต่อหน่วย ราคารวมสุทธิ (TOTAL DUE)
   - วิธีชำระเงิน จำนวนเงินที่รับ และเงินทอน
   - **QR Code บริเวณท้ายใบเสร็จ:** สำหรับให้ลูกค้าใช้กล้องมือถือสแกนเปิดอ่านใบเสร็จดิจิทัล
2. **รายงานสรุปยอดขาย (Sales Summary Report):**
   - **Filter Controls:** Dropdown เลือกสาขา (ทุกสาขา / B001 / B002 / B003) และตัวเลือกช่วงวันที่ (Start Date – End Date)
   - **KPI Cards:** บัตรสรุปตัวชี้วัดสำคัญ 3 ช่อง ได้แก่ จำนวนออเดอร์รวม, จำนวนแก้วที่ขายได้ และยอดขายรวม (บาท)
   - **Sales Data Table:** ตารางแสดงรายชื่อเมนู สาขา จำนวนที่ขาย และยอดขายรวม
3. **รายงานเปรียบเทียบระหว่างสาขา (Comparative Report):**
   - ตารางเปรียบเทียบประสิทธิภาพการขายระหว่างสาขา B001, B002, B003 เพื่อให้เจ้าของร้านเห็นสัดส่วนรายได้ชัดเจน

บทที่ 9: สถาปัตยกรรมระบบ (System Architecture)

9.1 รูปแบบสถาปัตยกรรมหลัก (3-Tier Layered Architecture)

ระบบถูกจัดโครงสร้างตามสถาปัตยกรรม 3 ชั้น เพื่อแยกหน้าที่ความรับผิดชอบ (Separation of Concerns):

1. **Presentation Layer (Frontend):** พัฒนาด้วย HTML5, CSS และ Vanilla JavaScript (app.js, *-ui.js) รันบนเว็บเบราว์เซอร์ของแท็บเล็ต POS ทำหน้าที่รับ Input และแสดงผล UI
2. **Application Layer (Backend):** พัฒนาด้วย Node.js และ Express.js จัดโครงสร้างตามรูปแบบ **MVC Pattern** ทำหน้าที่ประมวลผล Business Logic, Validation, การคำนวณเงิน, การจัดการคิว และ API Routes
3. **Data Layer (Database):** ระบบจัดการฐานข้อมูล MySQL ทำหน้าที่จัดเก็บข้อมูลโครงสร้าง Relational Data ตาม wk07-schema.sql สื่อสารผ่าน Database Driver (src/config/db.js)

9.2 การอธิบายตาม 4+1 View Model

1. Logical View (โมดูลการทำงาน)

- **Menu Management Module:** จัดการเมนูและสต็อก (menuController.js, menuItemModel.js)
- **Order & POS Module:** จัดการออเดอร์และการคิดเงิน (orderController.js, orderModel.js, Order.js)
- **Cashier & Receipt Module:** จัดการข้อมูลพนักงานและการออกใบเสร็จ (cashierModel.js, receipt-ui.js)
- **Report & Analytics Module:** คำนวณสรุปยอดขายตามสาขา (reportController.js, reportModel.js)

2. Development View (โครงสร้าง Directory จริงใน Repository)

```
None
cafe-pos-system/
├── public/                 # Frontend Static Assets
│   ├── index.html          # หน้าสั่งซื้อ POS หน้าร้าน
│   ├── index-ui.js         # Logic ฝั่ง Client หน้าสั่งซื้อ
│   ├── menu.html           # หน้าจัดการเมนูและสต็อก
│   ├── menu-ui.js          # Logic หน้าจัดการเมนู
│   ├── receipt.html        # หน้าแสดงและพิมพ์ใบเสร็จ
│   ├── receipt-ui.js       # Logic ดึงข้อมูลใบเสร็จ
│   ├── report.html         # หน้าแดชบอร์ดรายงานยอดขาย
│   └── report-ui.js        # Logic ประมวลผลรายงาน
├── src/                    # Backend Source Code (MVC)
│   ├── config/             # db.js (MySQL Connection)
│   ├── controllers/        # Business Logic (menu, order, report)
│   ├── models/             # Database Query Mapping Models
│   ├── routes/             # API Endpoints Router Setup
│   └── app.js              # Express Application Server Entry
├── wk07-schema.sql         # Database Schema Migration File
└── package.json            # Dependencies and Project Metadata
```

3. Process View (การสื่อสารระหว่างวัตถุ)

อธิบายการสื่อสารแบบ Asynchronous ผ่าน HTTP REST API:

- Client ส่ง POST /api/orders ⟶ orderRoutes.js ⟶ orderController.js ประมวล ผล Logic ⟶ orderModel.js ส่งคำสั่ง SQL Transaction ไปยัง MySQL DB ⟶ ตอบกลับ 201 Created ⟶ Client เปลี่ยนหน้าไป receipt.html ดึงข้อมูลผ่าน GET /api/reports/receipt/:id

4. Physical / Deployment View (โครงสร้างฮาร์ดแวร์และการติดตั้ง)

- **Client Device:** แท็บเล็ต Android หน้าร้านเปิด Chrome Browser โหลดไฟล์ Frontend จาก Web Server
- **Application Server:** Node.js Host รัน Express Web Server ที่พอร์ต 3000
- **Database Server:** MySQL Server จัดเก็บข้อมูลและสื่อสารผ่านพอร์ต 3306

บทที่ 10: ผลการทดสอบ Usability (Usability Testing)

10.1 วิธีการและสถานการณ์การทดสอบ

ทำการทดสอบแบบ **Think-Aloud Protocol** ร่วมกับผู้ทดสอบ เพื่อประเมินความพยายาม ความเข้าใจ และความสะดวกในการใช้งาน UI โดยมี 3 Tasks หลัก:

- **Task 1 (Happy Path):** สั่งซื้อ มอคค่า 1 แก้ว ปรับระดับความหวาน เพิ่มท็อปปิ้ง และยืนยันออเดอร์ชำระเงินสด
- **Task 2 (Happy Path):** ตรวจสอบสถานะเมนูที่หมด และทำการกดเพิ่มสต๊อกวัตถุดิบให้กลับมา พร้อมขาย
- **Task 3 (Error Path):** ลูกค้าสั่ง นมสด 2 แก้ว แต่สต๊อกในระบบคงเหลือเพียง 1 แก้ว ตรวจสอบการ ตอบสนองของระบบและการแจ้งเตือนแคชเชียร์

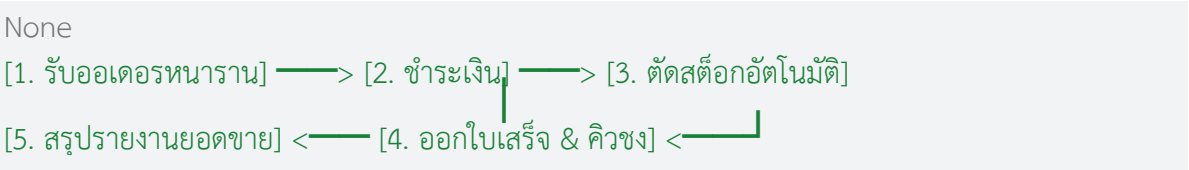
10.2 รายการ Usability Issues ที่พบและการแก้ไข

Issue #	รายละเอียดปัญหาที่พบ	Severity	หลักการ Usability ที่เกี่ยวข้อง	แผนการแก้ไข / การปรับปรุงที่ทำ
#1	เมื่อทดสอบเพิ่มจำนวนเบน 2 แก้วใน Task 3 ปุ่มกดไม่ตอบสนอง แต่ผู้ทดสอบชะงักและพูดว่า *"ทำไมกดเพิ่มจำนวนไม่ได้"* ก่อนจะเข้าใจเองว่าสต๊อกไม่พอ	Minor	**Feedback:** ระบบยังแสดงสาเหตุของการกดเพิ่มไม่ได้ไม่ชัดเจนในทันที	เพิ่มข้อความแจ้งเตือนป๊อปอัพทันที เมื่อผู้ใช้พยายามกดเพิ่มสินค้าเกินจำนวนสต๊อกที่มีอยู่จริง
#2	ใน Task 2 ขณะต้องการเพิ่มสต๊อก ผู้ทดสอบพูดว่า *"กดเพิ่ม Stock ตรงไหน"* ก่อนจะค้นพบปุ่มในหน้าจัดการเมนู	Minor	**Visibility:** ตำแหน่งและชื่อปุ่มคำสั่งเพิ่มสต๊อกยังไม่โดดเด่นในจุดที่สายตา	ปรับเปลี่ยนสีและตำแหน่งของปุ่มเพิ่มสต๊อกให้โดดเด่นขึ้น พร้อมปรับข้อความบนปุ่มให้ชัดเจน

บทที่ 11: ผลการรวมระบบ (System Integration)

11.1 การทดสอบการทำงานเชื่อมโยงแบบ End-to-End (E2E Integration Flow)

กลุ่มได้ทำการทดสอบการเชื่อมโยงระบบสมบูรณ์ ตั้งแต่ฝั่ง Frontend, Backend API, จนถึง Database Persistency โดยแบ่งเป็น 5 ขั้นตอนหลัก:



1. **สร้างออเดอร์ (Order Creation):** แคชเชียร์เลือกเมนูผ่าน index.html สคริปต์ index-ui.js ส่งข้อมูลเข้า POST /api/orders
2. **ประมวลผลและชำระเงิน (Payment Processing):** orderController.js ตรวจสอบความถูก ต้อง คำนวณเงินทอน บันทึกลงตาราง orders และ order_item
3. **ตัดสต็อกวัตถุดิบ (Stock Deduction):** ระบบรัน SQL Transaction สั่ง UPDATE menu_item SET stock_quantity = stock_quantity - ? โดยอัตโนมัติ
4. **ส่งคิวและออกใบเสร็จ (Queue & Receipt):** ระบบเปลี่ยนหน้าไปยัง receipt.html ดึงข้อมูล ผ่าน GET /api/reports/receipt/:id มาเรนเดอร์ใบเสร็จดิจิทัลพร้อม QR Code
5. **อัปเดตรายงานยอดขาย (Sales Report Integration):** เมื่อเปิดหน้า report.html สคริปต์ report-ui.js เรียก GET /api/reports/sales?branchId=... ดึงข้อมูลยอดขายที่ เพิ่งตัดบิลไปมาคำนวณรวมบนบัตร KPI และตารางสรุปยอดขายได้ถูกต้องเรียลไทม์

บทที่ 12: ผลการตรวจสอบคุณภาพและความปลอดภัย (Quality & Security Review)

12.1 สรุปผลการตรวจ Code & Security Checklist (12 API Endpoints)

หัวข้อการตรวจสอบ	**ผลการตรวจ**	**รายละเอียดและการป้องกัน**
1. SQL Injection	**ผ่าน**	ใช้ Parameterized Queries (mysql2 Prepared Statements ?) ในทุก Model endpoints ไม่มีการต่อ สตริง SQL ตรง
2. Race Condition สต็อก	**ผ่าน**	จัดทำ ACID Database Transaction และใช้คำสั่งล็อกแถวข้อมูลสต็อกขณะทำรายการตัดยอด ป้องกันสต็อกติดลบกรณีสั่งซื้อพร้อมกัน

หัวข้อการตรวจสอบ	ผลการตรวจ	รายละเอียดและการป้องกัน
3. Error Handling	ผ่าน	มีการล้อม try-catch บล็อกใน Controller และ Model ที่ติดต่อฐานข้อมูล พร้อมตอบกลับ Status Code เหมาะสม (เช่น 400, 500)
4. Input Validation	ผ่าน	มีการตรวจเช็กประเภทข้อมูล ค่าว่าง และช่วงตัวเลขของพารามิเตอร์ที่ส่งเข้ามาใน Request Body
5. Authorization / IDOR	ต้องปรับปรุง	**ความเสี่ยงสูง:** ปัจจุบันระบบยังไม่มีโครงสร้าง Authentication (Login/Session) ทำให้ผู้เรียก API สามารถส่ง branchId เพื่อดูหรือแก้ข้อมูลข้ามสาขาได้

12.2 การแก้ไขความปลอดภัยที่ได้ดำเนินการแล้ว

- **การป้องกัน Cashier ผิดสาขา:** ได้ทำการเพิ่มการตรวจสอบใน orderController.js เพื่อเช็กว่า cashier_id ที่ส่งมา สังกัดอยู่ใน branch_id เดียวกับออเดอร์นั้นจริงหรือไม่ หากไม่ตรงกัน ระบบจะปฏิเสธคำขอและสั่ง Rollback Transaction ทันที ช่วยป้องกันการผูกยอดขายผิดสาขา

บทที่ 13: สรุปผลและข้อเสนอแนะ (Conclusion & Recommendation)

13.1 สรุปผลการดำเนินโครงการ

การพัฒนาระบบ **Buumonkey Cafe POS & Inventory System** บรรลุวัตถุประสงค์หลักตามที่ได้วางไว้ ระบบสามารถช่วยให้พนักงานแคชเชียร์บันทึกออเดอร์และคิดเงินได้อย่างถูกต้องรวดเร็ว ตัดสต็อกวัตถุดิบอัตโนมัติตามสูตรเมนูทันทีที่มีการขาย ออกใบเสร็จดิจิทัลผ่าน QR Code และจัดทำรายงานสรุปยอดขายเปรียบเทียบระหว่าง 3 สาขาให้เจ้าของร้านได้แบบ Real-time ช่วยลดเวลาการปิดยอดบัญชีประจำวันจาก 30–45 นาที เหลือเพียงไม่กี่นาที และลดข้อผิดพลาดจาก Human Error ได้อย่างมีประสิทธิภาพ

13.2 บทเรียนที่ได้รับ (Lessons Learned)

1. **ประโยชน์ของการออกแบบตามสถาปัตยกรรม MVC:** การแยกโค้ดออกเป็น Controller, Model และ Routes ชัดเจน ช่วยให้สมาชิกในทีมสามารถแบ่งงานกันพัฒนาได้โดยโค้ดไม่ขัดแย้งกัน (Conflict)
2. **ความสำคัญของ Price Snapshotting:** การบันทึกราคาลงใน order_item.unit_price ณ เวลาสั่งซื้อ ถือเป็นหัวใจสำคัญในการรักษาความถูกต้องของข้อมูลบัญชียอดขายย้อนหลัง

3. **การสื่อสารผ่าน API Contract:** การกำหนดโครงสร้าง JSON Payload (Naming Conventions เช่น camelCase vs snake_case) ร่วมกันล่วงหน้า ช่วยลดเวลาในการ Debug ตอนรวมระบบ (Integration) ได้อย่างมาก

13.3 ข้อจำกัดและแนวทางการพัฒนาต่อในอนาคต (Future Work)

1. **การพัฒนาระบบ ยืนยันตัวตนและกำหนดสิทธิ์ (Authentication & Authorization):** เพิ่มระบบ Login ด้วย JWT (JSON Web Token) หรือ Session เพื่อจำกัดสิทธิ์การเข้าถึงข้อมูลของแคชเชียร์ แต่ละสาขา และป้องกันการเข้าถึง API โดยไม่ได้รับอนุญาต
2. **การเชื่อมต่อ Payment Gateway จริง:** พัฒนาการเชื่อมต่อกับ PromptPay API ของสถาบันการเงิน เพื่อตรวจสอบการชำระเงินผ่าน QR Code แบบอัตโนมัติ (Auto Verification)
3. **ระบบสมาชิกและสะสมแต้ม (CRM Loyalty Program):** เพิ่มฟีเจอร์สะสมแต้มผ่านเบอร์โทรศัพท์ลูกค้าเพื่อนำมาแลกส่วนลดในครั้งถัดไป

บทที่ 14: ภาคผนวก (Appendix)

14.1 รายงานการใช้งานปัญญาประดิษฐ์ (AI Disclosure Summary Report)

ตลอดภาคการศึกษา กลุ่ม buumonkey ได้ใช้งานเครื่องมือ AI (GitHub Copilot, ChatGPT, Gemini, Claude) ในฐานะผู้ช่วยผู้พัฒนาระบบ (AI Assistant) โดยสมาชิกในกลุ่มเป็นผู้ตรวจสอบ ตัดสินใจ และทดสอบระบบจริงด้วยตนเองทุกครั้ง สรุปรายละเอียดได้ดังนี้:

1. Coding Sprint สัปดาห์ที่ 5 (wk05-ai-disclosure.md)
- **งานที่ใช้:** ช่วยร่างฟังก์ชัน createOrder ใน orderController.js
- **สมาชิกผู้รับผิดชอบอธิบายโค้ด:** จิรภัทร บัวชุมสุข

2. Coding Sprint สัปดาห์ที่ 9 (wk09-ai-disclosure.md)
- **เครื่องมือ:** GitHub Copilot, ChatGPT
- **งานที่ใช้:** ตรวจสอบ Diagram-Code Drift, ช่วยปรับปรุง Schema ตาราง menu_item, orders, order_item และตรวจสอบโครงสร้าง MVC
- **สมาชิกผู้รับผิดชอบอธิบายโค้ด:** จิรภัทร บัวชุมสุข, ณฐกร โกศินานนท์

3. Requirement & Design Sprint สัปดาห์ที่ 10 (wk10-ai-disclosure.md)
- **เครื่องมือ:** Claude
- **งานที่ใช้:** ช่วยร่างเอกสาร SRS, Wireframe หน้า POS, ใบเสร็จดิจิทัล และ Sequence Diagram ของ endpoint รายงานยอดขาย

4. Integration Sprint สัปดาห์ที่ 13 (wk13-ai-disclosure.md)
- **เครื่องมือ:** ChatGPT, Gemini

- **งานที่ใช้:** ออกแบบ E2E Test Cases, ตรวจสอบ API Naming Mismatch, ร่างโค้ดโมดูลรายงานสรุปยอดขายแยกตามสาขา (reportController.js, reportModel.js)
- **สมาชิกผู้รับผิดชอบอธิบายโค้ด:** จิรภัทร บัวชุมสุข, ณฐกร โกศินานนท์

หมายเหตุ: เอกสาร AI Disclosure Form ฉบับเต็มของแต่ละสัปดาห์ ถูกจัดเก็บไว้ในโฟลเดอร์ docs/ai-disclosure/ บน GitHub Repository ของกลุ่ม